Quantum Eigenvalue Solver

Team : Undying Cat

Authors : Seng, Z. Y., Tan, K. M., Leong, K. F., Lau, Z. H., Tan, J. B.

Objective

Develop a general solver to numerically determine all energy levels by giving an arbitrary potential $V(x)$.

1 Introduction

1.1 Time-Independent Schrödinger equation

The time-independent Schrödinger equation (TISE) in one dimension is given by

$$-\frac{\hbar^2}{2m}\frac{d^2\psi(x)}{dx^2} + V(x)\psi(x) = E\,\psi(x) \tag{1}$$

This equation is a second-order linear ordinary differential equation, where each solution $\psi_n(x)$ corresponds to an eigenfunction with an associated energy eigenvalue E_n. The general solution $\Psi(x)$ can be expressed as a linear combination of these eigenfunctions.

However, for arbitrary potential functions $V(x)$, obtaining analytical solutions for $\psi(x)$ and $E_n$is often difficult and, in many cases, impossible without approximation methods such as series expansions.

In many applications, only the discrete energy eigenvalues E_n are of primary interest. Therefore, instead of solving the equation analytically, a numerical approach can be employed. By reformulating the problem as an initial value problem, numerical methods can be used to efficiently determine the allowed energy levels. In which, we call as the **shooting method.**

1.2 Runge–Kutta–Fehlberg method (RK45)

The Runge–Kutta–Fehlberg method (RK45) is an adaptive numerical method other than Euler's method used to solve ordinary differential equations of the form:

$$\frac{du}{dx} = f(x,u), u(x_0) = u_0$$

It advances the solution step-by-step by estimating it using two different orders of accuracy (4th and 5th order). The difference between these two estimates is used to measure the numerical error.

For each step, RK45 computes several intermediate slopes $k_1, k_2, \dots, k_6$ with arbitrary small steps h and then forms $u_{n+1} = u(x_o + (n+1)h)$ in terms of RK4 approximation and RK5 approximation

$$u_{n+1}^{(RK4)} = u_n + h\left(\frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5\right)$$

$$u_{n+1}^{(RK5)} = u_n + h\left(\frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6\right)$$

The error is estimated as:

$$\epsilon = | u_{n+1}^{(RK5)} - u_{n+1}^{(RK4)} |$$

This error is compared with a tolerance:

$$\text{tol} = \text{atol} + \text{rtol} \times |u|$$

If $\epsilon <$ tol, the step is accepted.
If not, the step is rejected and the step size is adjusted:

$$h_{\text{new}} = h\left(\frac{\text{tol}}{\epsilon}\right)^{1/5}$$

This adaptive control ensures both accuracy and efficiency.

In Python, we can call the RK45 solver using **solve_ivp** from **scipy.integrate:**

```
from scipy.integrate import solve_ivp
def f(x, u):
    return ...

sol = solve_ivp(fun=f, t_span=[x0,…,xf], y0=[u0], method='RK45', atol=1e-8,
rtol=1e-8)

>>> sol will return to [u0,. . .,uf] corresponding [x0,...,xf]
```

2 Methodology

2.1 Using RK45 (Shooting Method) to solve TISE numerically

To solve out the equation (1) numerically, we rearrange it to get

$$\frac{d^2\psi(x)}{dx^2} = \frac{2m}{\hbar^2}[V(x) - E]\psi(x) \tag{2}$$

Letting $u(x) = \psi(x)$and $y(x) = \frac{du}{dx}$. We can construct vector $\boldsymbol{U}$

$$\boldsymbol{U} = \begin{pmatrix} u(x) \\ y(x) \end{pmatrix}$$

Thence, the TISE is reduced to solving a system of first-order ODEs with 2 components over domain $[a, b] \approx [-\infty, +\infty]$

$$\boldsymbol{U'} \equiv \begin{pmatrix} f(x,u) \\ f(x,y) \end{pmatrix} = \begin{pmatrix} \dfrac{du}{dx} \\ \dfrac{dy}{dx} \end{pmatrix} = \begin{pmatrix} y(x) \\ \dfrac{2m}{\hbar^2}[V(x) - E]u(x) \end{pmatrix}$$

with initial condition

$$\boldsymbol{U_o} = \begin{pmatrix} u'(a) \\ \dfrac{2m}{\hbar^2}[V(x) - E]\, u(a) \end{pmatrix}$$

Hence in Python you will see

```
# Define func with two components
def func(x, u_vec, E):
    u, du = u_vec
    d2u = (2.0 * m / hbar**2) * (V(x) - E) * u
    return [du, d2u]
```

2.2 Setting initial condition

a is chosen at the left (negative) boundary where $a \to -\infty$, the wavefunction is expected to decay to zero in that region. Hence, we impose the initial condition

$$u(a) = 0$$

because $u(x) = \psi(x)$ approximately vanishes at sufficiently large negative x.

For the derivative, we set a convenient non-zero value, for example

$$u'(a) = 1$$

This choice is arbitrary in magnitude because the Schrödinger equation is linear, so the overall amplitude of the solution will later be adjusted by normalization. What matters physically is the shape of the wavefunction, not its initial scale. Setting $u'(a) = 0$ is avoided because it would produce the trivial solution $u(x) = 0$ everywhere.

Hence, whole shooting method to approximate $\psi(x)$ is given below :

```
import numpy as np
from scipy.integrate import solve_ivp
def V(x):
    return 0.5 m * w * x **2 # eg. harmonic oscillator

def func(x, u_vec, E):
    u, du = u_vec
    d2u = (2.0 * m / hbar**2) * (V(x) - E) * u
    return [du, d2u]

def shoot(E):
```

```
    sol = solve_ivp(fun=func, t_span=(-L, L), y0=[0, 1], method="RK45",
args=(E,), atol=1e-8, rtol=1e-8)

# u_final = u(L)
    u_final = sol.y[0, -1]

# if |u_final|>1e10, return u_final to np.sign(u_final) * 1e10 to avoid overflow
    return np.sign(u_final) * min(abs(u_final), 1e10)
```

2.3 First Idea to solve out $\psi(x)$ and E

1. **Guess an energy value E**
   We start by assuming a trial value of E as the eigenvalue.
2. **Integrate the wavefunction numerically**
   Using the guessed E, we integrate $u(x)$ by RK45 over the domain :

$$[a, b] \approx [-\infty, +\infty]$$

3. **Check boundary condition at $x_{final} = b$**
   If the solution satisfies $u(b) \approx 0$, then the chosen E is a valid eigenvalue, and the $u(x)$ is the corresponding eigenfunction of the TISE. Otherwise, adjust E and repeat the until convergence.

However, the idea above encounters a problem: the function $u(x)$ often diverges even when the guessed energy $E_{\text{guess}} \approx E_{\text{exact}}$. This instability can be understood as follows:

2.4 Prove guess solution $u(x)$ diverges to infinity as E_{guess} not exactly equal to E_{exact}

Start from equation (2)

For exact solution $\psi(x)$

$$\frac{d^2\psi(x)}{dx^2} = \frac{2m}{\hbar^2}[V(x) - E_{exact}]\psi(x)$$

For guess solution $u(x)$

$$\frac{d^2u(x)}{dx^2} = \frac{2m}{\hbar^2}\left[V(x) - E_{guess}\right]u(x)$$

Multiply the first by $u(x)$ and second by $\psi(x)$ we obtained

$$u(x)\frac{d^2\psi(x)}{dx^2} = \frac{2m}{\hbar^2}[V(x) - E_{exact}]u(x)\psi(x) \tag{3}$$

$$\frac{d^2u(x)}{dx^2}\psi(x) = \frac{2m}{\hbar^2}\left[V(x) - E_{guess}\right]u(x)\psi(x) \tag{4}$$

Extract (3) with (4)

$$u(x)\frac{d^2\psi(x)}{dx^2}-\frac{d^2u(x)}{dx^2}\psi(x)=\frac{2m}{\hbar^2}\left[E_{guess}-E_{exact}\right]u(x)\psi(x)$$

Rewrite $\delta E=E_{guess}-E_{exact}$ and $\frac{d}{dx}\left(u(x)\frac{d\psi}{dx}-\psi(x)\frac{du}{dx}\right)=u(x)\frac{d^2\psi(x)}{dx^2}-\frac{d^2u(x)}{dx^2}\psi(x)$:

$$\frac{d}{dx}\left(u(x)\frac{d\psi}{dx}-\psi(x)\frac{du}{dx}\right)==\frac{2m}{\hbar^2}\delta E\,u(x)\psi(x)$$

Take integration over $[a,b]$:

$$[u(x)\psi'(x)-\psi(x)u'(x)]_{\mathrm{a}}^{b}=\frac{2m}{\hbar^2}\delta E\int_a^b u(x)\psi(x)dx$$

Apply initial condition $u(a)=0\ ,\ u'(a)=1$ gives

$$u(b)\psi'(a)-u'(a)\psi(a)-\psi(a)=\frac{2m}{\hbar^2}\delta E\int_a^b u(x)\psi(x)dx$$

Since bound states behaves asymptotically with $\boldsymbol{\psi(x)\sim Ae^{-\kappa|x|}}$ for large $|x|$ (WKB idea in the classically forbidden region $V(x)>E$), where $\kappa=\frac{\sqrt{2m(V(x)-E)}}{\hbar}$. The wavefunction decays exponentially as the potential dominates the energy, and the solution remains **normalizable at infinity.**

Substitute $\psi(x)\sim Ae^{-\kappa|x|}$ and $\psi'(x)\sim-\kappa Ae^{-\kappa|x|}$ on LHS :

$$u(b)(-\kappa Ae^{-\kappa b})-u'(b)Ae^{-\kappa b}-Ae^{-\kappa b}=\frac{2m}{\hbar^2}\delta E\int_a^b u(x)\psi(x)dx$$

To simplify the expression and our algorithm we will use $\boldsymbol{a=-L}$ **while** $\boldsymbol{b=+L}$**.** We will show it is appropriate if we have to take L large (you might think of it is a radius of convergence). The expression becomes

$$u(L)(-\kappa Ae^{-\kappa L})-u'(L)Ae^{-\kappa L}-Ae^{-\kappa L}=\frac{2m}{\hbar^2}\delta E\int_{-L}^{L}u(x)\psi(x)dx$$

Simplify to get

$$u(L)+\frac{u'(L)}{\kappa}=-\frac{2m}{\hbar^2\kappa A}\int_{-L}^{L}u(x)\psi(x)dx\cdot\delta E\cdot e^{\kappa L}+1$$

Since $\int_{-L}^{L}u(x)\psi(x)dx$ must be finite (i.e. $\psi(x)$ is bounded), we rewrite it as $I(L)$

$$u(L)+\frac{u'(L)}{\kappa}=-\frac{2m}{\hbar^2\kappa}\frac{I(L)}{A}\delta E\cdot e^{\kappa L}+1$$

We can see that when δE is not exactly zero, the LHS will just blows up to infinity as L approaches to infinity. To see the $u(L)$ directly, we may assume that when $|x|$ is large, the $u(x)$ reads

$$u(x) \sim Ce^{-\kappa x} + De^{\kappa x}$$

Substitute $x = L$ gives $u(L) \sim Ce^{-\kappa L} + De^{\kappa L}$ and $u'(L) \sim -\kappa Ce^{-\kappa L} + \kappa De^{\kappa L}$ gives

$$2De^{\kappa L} = -\frac{2m}{\hbar^2 \kappa}\frac{I}{A} \cdot \delta E \cdot e^{\kappa L} + 1$$

Since scalar +1 is negligible to $e^{\kappa L}$ and substitute back $u(L) \sim Ce^{-\kappa L} + De^{\kappa L}$

$$\Longrightarrow u(L) \approx -\frac{m\, I(L)}{\hbar^2 \kappa A} \cdot \delta E \cdot e^{\kappa L} \tag{5}$$

We don't know sign of $I(L)$ and A, it's better to take absolute value before taking logarithm :

$$\ln|u(L)| = \kappa L + \ln\left|\frac{m\, I(L)}{\hbar^2 \kappa A} \cdot \delta E\right|$$

Because $I(L)$ is nearly constant when L are just slightly different $L_1 \approx L_2 \approx L_3 \approx \cdots$

$$\ln|u(L)| = \kappa L + C \tag{6}$$

This linear relation between and $\ln|u(L)|$ and L holds for different values $L_1, L_2, L_3, \ldots$that are sufficiently large and close to each other. (Here, "L is sufficiently large" introduces the concept of an "optimal L," which will be discussed later in Section 2.6)

2.5 Find approximate eigenvalues E instead of finding approximate eigenfunction $u(x)$

From equation (5), even when $E_{\text{guess}} \approx E_{\text{exact}}$, the approximate solution $u(x)$still diverges to $\pm\infty$ due to the exponentially growing term $e^{\kappa L}$ at large L. As a result, $u(x)$ cannot be normalized, and the desired eigenfunctions are nearly impossible to get.

Nevertheless, finding eigenvalues E_{exact} is still possible by applying root-finding method**.** Assume we have an optimal L , and we test two guess energies E_{low} and E_{high}. If the approximate solutions $u(L\,; E_i)$ gives different signs (e.g. $u_{E_{low}}(L) < 0$ and $u_{E_{high}}(L) > 0$) The exact eigenvalue E_{exact} must located in between E_{low} and E_{high}. This allows us to apply the **bisection method** to iteratively narrow down the E_{low} E_{high} and converge to the correct eigenvalue. To mathematically proof, set :

$$F(E) = u(L\,; E)$$

Suppose $F(E)$ is continuous, $F(E_{low})$ and $F(E_{high})$ gives different signs. From Intermediate Value Theorem there must exist $\exists E_{exact}$ within $(E_{low}\,, E_{high})$ where $F(E_{exact}) = 0$.

To improve both speed and accuracy, we replace the pure bisection method with **Brent's method**, which combines bisection, secant, and inverse quadratic interpolation for faster and

more reliable convergence. In Python, we search for a root by choosing an energy interval $[E_{\text{min}}, E_{\text{max}}]$, and testing pairs $(E_{\text{low}}, E_{\text{high}})$ within this range to locate a sign change, which is then using Brent's Method to converge efficiently to the eigenvalue $E_{root} \approx E_{exact}$

```
from scipy.optimize import brentq

def find_eigenvalues(E_min, E_max, nsteps, shoot=shoot):
    energies = np.linspace(E_min, E_max, nsteps)
    eigenvalues = []

    # Calculate the first shot
    f_old = shoot(energies[0])

    for i in range(len(energies) - 1):
        E_low = energies[i]
        E_high = energies[i+1]
        f_new = shoot(E_high)

        # Sign change detected -> root found in this interval
        if f_old * f_new < 0:
            E_root = brentq(shoot, E_low, E_high)
            eigenvalues.append(E_root)

        f_old = f_new

    print(eigenvalues)

    if not eigenvalues:
        print("No eigenvalues found. Try different L or E_min, E_max.")

find_eigenvalues(E_min=0, E_max=5, nsteps=100) # Example
```

2.6 Finding an "optimal" L

There is still one question left: how do we choose the "radius of convergence" L such that integrating $u(x)$ over $x_{span} = [-L, L]$ gives a reliable eigenvalue E?

In practice, we can set $x_{\text{span}} = [-L, L]$ and perform the RK45 integration together with Brent's method to determine the ground state energy. We then increase L progressively and repeat the calculation. If the obtained ground state eigenvalue E_{ground} remains stable (unchanged within tolerance) for consecutive values of L, and no numerical overflow occurs, we regard that value of L as the optimal choice L_{optimal}, since it is sufficiently large to approximate the infinite domain while still numerically stable on predicting E.

2.7 Finalised Algorithm

Thence, the steps to find eigenvalues E by giving an arbitrary potential $V(x)$ are as follows :

i. Import libraries and assigning m and $\hbar$:

```
import numpy as np

from scipy.integrate import solve_ivp
from scipy.optimize import brentq

m, hbar = 1.0, 1.0 # natural units
```

ii. Define $V(x)$, `shoot()`a in Example 1: Google Colab | GitHub

```
def V(x): # for example harmonic oscillator
    return 0.5 * x **2
```

iii. Guess a range of E_{min} and E_{max} . For example, 0.0 to 10.0 (natural units) if you know:

```
E_min, E_max = 0.0, 10.0
```

iv. Define `find_optimalL()`. Guess a range of L_{min} and L_{max} then run the `find_optimalL()` to get an optimal L without having overflow. If the code doesn't get optimal L, you might change the range of L_{min} and L_{max}.

```
# Change L_min, L_max, and nsteps here e.g. Lmin = 1, Lmax=20, nsteps = 2
    for L in np.arange(2, 10, 1):
    ......
```

In example above the Python will check $L_{optimal}$ from 2 to 10 by increasing 1.

v. Repeating iv. to get $L_{optimal}$

```
L          | E_root
------------------------
2.0        | 0.53746121
3.0        | 0.50039108
4.0        | 0.50000049
5.0        | 0.50000000
------------------------
Stability reached at L = 5.0
```

vi. Define and run the find_eigenvalues() with $L_{optimal}$

```
L = 5.0
find_eigenvalues(E_min, E_max, n=100)
```

vii. Obtain the E_{root}

```
[0.49999999995968086, 1.5000000033667265, 2.5000000835230987,
```

Which corresponds to eigenvalues E_0, E_1, E_2, ... from E_{min} to E_{max}

3 Results

We test several potentials $V(x)$ with natural units $\hbar = m = 1$, including :

3.1 Harmonic Oscillator

$$V(x) = \frac{1}{2} m\omega^2 x^2$$

$$E_n = \hbar\omega\left(n + \frac{1}{2}\right)$$

Outcome

Click to see the code : Google Colab | GitHub

The eigenvalues E when substituting $\hbar = m = \omega = 1$ are

```
[0.4999999999596808, 1.5000000033667265, 2.5000000835230987, 3.5000012207653235, 4.5000126363467805]
```

highly aligns to theoretical results.

3.2 Modified Pöschl–Teller Potential

$$V(x) = -\frac{V_0}{cosh^2(\alpha\, x)}$$

$$E_n = -\frac{\hbar^2\alpha^2}{2m}\left[\sqrt{\frac{2mV_0}{\hbar^2\alpha^2} + \frac{1}{4}} - \left(n + \frac{1}{2}\right)\right]^2$$

Outcome

Click to see the code : Google Colab | GitHub

The eigenvalues E when substituting $\hbar = m = 1, V_0 = 6, \alpha = 0.5$ are

```
[-5.194222250531993, -3.70766664790057, -2.4710987599131977, -1.4839171028267653, -0.7347190023107398,...]
```

highly aligns to theoretical results.

3.3 Morse potential

$$V(x) = D_e(1 - e^{-ax})^2$$

$$E_n = \hbar a\sqrt{\frac{2D_e}{m}}\left(n + \frac{1}{2}\right) - \frac{\hbar^2 a^2}{2m}\left(n + \frac{1}{2}\right)^2$$

Outcome

Click to see the code : Google Colab | GitHub

The eigenvalues E when we substitute $\hbar = m = 1, D_e = 8, a = 0.5$ are

```
[0.9687499998338143, 2.7187500004342935, 4.21875073710904]
```

highly aligns to theoretical results.

3.4 Hydrogen Atom Effective Potential

$$V_{\text{eff}}(r) = -\frac{e^2}{r} + \frac{1}{4\pi\epsilon_o}\frac{\hbar^2 l(l+1)}{2mr^2}$$

$$E_n = -\frac{mZ^2e^4}{2\hbar^2 n^2}$$

Outcome

Click to see the code : Google Colab | GitHub

The eigenvalues E when we substitute $\hbar = m = e = \frac{1}{4\pi\epsilon_o} = 1$, $l = 2$ are

```
[-0.05555553891597539, -0.031174611130845777, -0.017190161808121465]
```

highly aligns to theoretical results.

Therefore, the eigenvalue solver has successfully solved the Harmonic Oscillator, Modified Pöschl-Teller, Morse potential, and Hydrogen atom effective potential with results consistent with analytical solutions.

4.5 Verify Equation (6)

To verify the validity of Equation (6),

$$\ln\ |u(L)| = \kappa L + C$$

we test whether a linear relationship exists between $\ln\ |u(L)|$ and L for sufficiently large and closely spaced values of L.

We consider the harmonic oscillator with potential

$$V(x) = \frac{1}{2}x^2,$$

in natural units $\hbar = m = \omega = 1$. The exact ground state energy for this system is $E = 0.5$, so we slightly perturb it to $E = 0.50001$ to simulate a non-exact eigenvalue, which is necessary for the shooting method to produce a non-zero boundary value.

The numerical procedure evaluates the wavefunction $u(L)$ at different values of L, defined as:

$$L = L_0 + \delta L,$$

with $L_0 = 5$ and small increments $\delta L \in [1, 2, 3, 4, 5, 6] \times 10^{-3}$.

For each value of L, the wavefunction is computed using the shooting method, and the value of $\ln\ |u(L)|$ is recorded. A linear regression is then performed on the dataset $(L, \ln\ |u(L)|)$.

Outcome

Click to see the code : Google Colab | GitHub

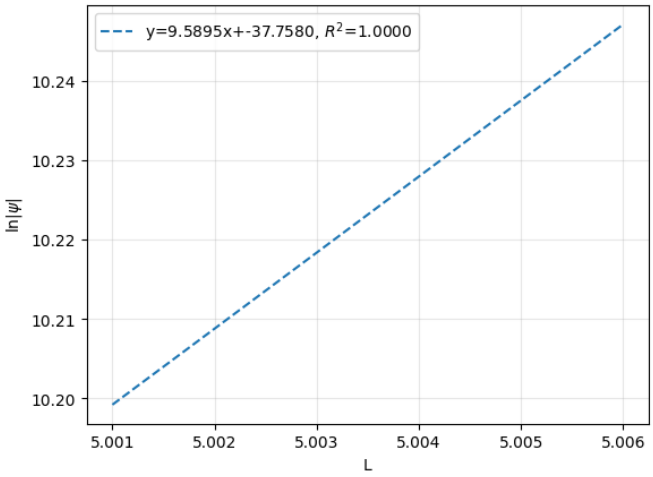

Figure 1 : Relation of ln $|u(L)|$ with L

This agrees the equation (6) and shows $\kappa \sim \frac{\sqrt{2m(V(x)-E)}}{\hbar}$ is a positive value.

4 Discussion

4.1 Alignment to theoretical results

The numerical eigenvalue solver developed in this study is able to successfully determine the energy eigenvalues for a variety of quantum systems. The results obtained for the harmonic oscillator, Morse potential, modified Pöschl–Teller potential, and hydrogen atom effective potential show strong agreement with known analytical solutions. This demonstrates that the combination of the shooting method, RK45 integration, and Brent's root-finding algorithm is both accurate and reliable for solving the time-independent Schrödinger equation.

The validity of equation (6) is supported by the strong alignment between the theoretical prediction and numerical results. The expected linear relationship between ln $|u(L)|$ and L is clearly observed for sufficiently large and closely spaced values of L, confirming that the numerical solution exhibits exponential behaviour in the asymptotic region.

$$\ln |u(L)| = \kappa L + C$$

This agreement verifies that the divergence of $u(L)$ is governed by the factor $e^{\kappa L}$, as derived theoretically in section 2.4. The consistency between simulation and theory not only validates the correctness of the derivation but also reinforces the reliability of the numerical method in capturing the expected asymptotic behaviour, despite the inherent instability of the shooting method for non-exact eigenvalues.

4.2 Limitation and unexpected outcomes

However, the applicability of the solver is subject to several important limitations. Firstly, the method is only valid for smooth potentials $V(x)$. Since the RK45 integrator assumes continuity and differentiability of the function, potentials with discontinuities or sharp boundaries (e.g., step potentials or infinite square wells) may lead to numerical instability or inaccurate results. This restricts the generality of the solver when dealing with non-smooth or piecewise-defined systems.

Another key limitation arises from the difficulty in directly solving for the wavefunction at the boundary $\psi(L)$. As shown in the analysis, even a small deviation between the guessed energy and the exact eigenvalue leads to exponential divergence of the numerical solution at large L.This makes it practically impossible to obtain a properly normalized eigenfunction using the shooting method alone. Instead, the method is more suitable for identifying eigenvalues through sign changes rather than constructing accurate wavefunctions.

Due to this instability, alternative numerical approaches may be required if the goal is to compute well-behaved eigenfunctions. Methods such as the finite difference method, Numerov algorithm, or matrix diagonalization techniques could provide improved stability and allow direct enforcement of boundary conditions.

Finally, the accuracy of the method depends strongly on the choice of parameters such as the domain size L, step tolerance, and energy search interval. An inappropriate choice of L may either introduce numerical overflow or fail to approximate the infinite domain adequately. Therefore, careful tuning of these parameters is essential to ensure convergence and reliability of the results.

4.3 Further Task

In addition, once a small set of eigenvalues is obtained, spectral patterns such as spacing regularity or asymptotic scaling can be analysed to construct an empirical energy model $E(n)$. For certain potentials (e.g., harmonic oscillator-like systems), this relationship may approach a linear or quadratic dependence on n, allowing efficient prediction of higher states without full numerical integration.

Furthermore, the present solver framework can be extended to a broader class of Sturm–Liouville eigenvalue problems beyond the specific quantum mechanical systems considered in this study. By reformulating the governing differential equation into a general eigenvalue form with variable coefficient functions and appropriate boundary conditions, the same numerical approach can be applied to other physical systems such as optical waveguides, vibrating mechanical structures with non-uniform density, and periodic lattice models in solid-state physics. This generalisation enables the study of how changes in system parameters influence eigenvalue spectra across different domains, while also providing a unified computational method for analysing bound-state, resonant, and mode-propagation problems within a consistent mathematical framework.

5 Conclusion

In this study, a numerical eigenvalue solver was successfully developed to solve energy levels for arbitrary smooth potential $V(x)$ based on shooting method and Brent's method. The solver accurately determines energy eigenvalues for several benchmark systems, including the harmonic oscillator, Morse potential, modified Pöschl–Teller potential, and hydrogen atom effective potential, with results showing strong agreement with known analytical solutions. This demonstrates the effectiveness and reliability of the implemented approach for eigenvalue determination.

However, several limitations were identified. The method requires smooth potentials and suffers from numerical instability due to the exponential divergence of the wavefunction when the trial energy deviates slightly from the exact eigenvalue. Consequently, the solver is more suitable for determining eigenvalues rather than constructing accurate eigenfunctions. In addition, the choice of the domain size Lplays a critical role in the accuracy and stability of the results. If $L$is too small, the system fails to approximate the infinite boundary conditions properly; if L is too large, numerical overflow may occur due to exponential growth. Therefore, selecting an optimal L is essential to ensure convergence and reliable eigenvalue estimation.

Overall, the developed solver provides a robust and efficient framework for studying quantum systems numerically. To further test its strength and reliability, more different types of potentials can be explored in future work. This would help confirm the robustness of the algorithm across a wider variety of quantum systems and increase confidence in its general applicability such as solid-state physics or material sciences.